UVSim

Design Document

Project Overview:

UVSim is a software simulator designed to help computer science students learn more about machine language and computer architecture. Students write programs in BasicML, a simple machine language, and run them on the simulator. The simulator includes a CPU, an accumulator register, and a 100-word addressable memory, with each word being a signed four-digit decimal number. The programs load into main memory starting at location 00 before running, and they support branching and arithmetic operations.

User Stories:

Running a BasicML Program

"As a computer science student, I want to load a BasicML program from a text file and execute it on the UVSim simulator, so that I can observe how machine language instructions work and learn computer architecture concepts."

Interacting with a Running Program

"As a computer science student, I want to provide input values during program execution and see the output printed to my screen, so that I can test my BasicML programs with different data and verify they produce correct results."

Interaction with GUI

"As a user needing accessibility options, I want to be able to change the color theme of the program, so that I can still read and understand the interface."

Use Cases:

1. Accept a .txt file

Actor – Student

Goal – Have the user give the simulator a file that can then be loaded.

Steps:

1. Student types python3 main.py and hits Enter
2. The simulator loads the graphical user interface
3. The student then can select 'load' to load a file and choose their file from any CD and open it
4. The simulator stores the filename for use in the next step

2. Read and Parse a BasicML File into GUI

Actor – Program

Goal – All instructions from the BasicML file have been parsed and stored in memory, ready for execution.

Steps:

1. The simulator opens the file
2. Each line is read one at a time
3. The sign, opcode, and operand are all parsed from each line
4. The parsed instruction is stored into the gui interface at the current index
5. The index increments by 1
6. Steps 2-5 repeated until all lines are loaded

7. When the user selects run, the file is then loaded to memory and is run

3. Prompt the User for Keyboard Input

Actor – Student

Goal – A valid integer value entered by the student has been validated and stored at the correct memory address.

Steps:

1. The CPU reads opcode 10 and the memory address operand
2. The program has a pop-up prompt asking the student to enter a value
3. The student types a signed integer and presses Enter or OK
4. The program validates the input is a proper int
5. The program validates the value is within the allowed range
6. The value is stored at the specified memory location
7. The program counter increments by 1

4. Display a Value from Memory to screen

Actor – Program

Goal – The value at the specified memory address has been formatted and printed to the GUI Output.

Steps:

1. The CPU reads opcode 11 and the memory address operand
2. The value at the specified memory location is retrieved
3. The value is formatted as a signed 6-digit number

4. The formatted value is printed to the output in the GUI
5. The program counter increments by 1

5. Add a Memory Value to the Accumulator

Actor – CPU

Goal – The accumulator holds the sum of its previous value and the value retrieved from the specified memory address.

Steps:

1. The CPU reads opcode 30 and the memory address operand from the current instruction
2. The value at the specified memory location is retrieved
3. That value is added to the current accumulator value
4. The result is stored back in the accumulator
5. The program counter increments by 1

6. Subtract a Memory Value from the Accumulator

Actor – Program

Goal – The accumulator holds the result of subtracting the value at the specified memory address from its previous value.

Steps:

1. The CPU reads opcode 31 and the memory address operand
2. The value at the specified memory location is retrieved
3. That value is subtracted from the current accumulator value

4. The result is stored back in the accumulator
5. The program counter increments by 1

7. Multiply the Accumulator by a Memory Value

Actor – Program

Goal – The accumulator holds the product of its previous value and the value retrieved from the specified memory address.

Steps:

1. The CPU reads opcode 33 and the memory address operand
2. The value at the specified memory location is retrieved
3. That value is multiplied by the current accumulator value
4. The result is stored back in the accumulator
5. The program counter increments by 1

8. Divide the Accumulator by a Memory Value

Actor – Program

Goal – The accumulator holds the integer quotients of its previous value divided by the value at the specified memory address.

Steps:

1. The CPU reads opcode 32 and the memory address operand
2. The value at the specified memory location is retrieved
3. The value is checked to ensure it is not zero
4. The accumulator is divided by that value using integer division

5. The result is stored back in the accumulator
6. The program counter increases by 1

9. Branch to a Specific Memory Location

Actor – Program

Goal – The program counter has been set to the target address, redirecting execution to that location.

Steps:

1. The CPU reads opcode 40
2. The program counted is set directly to the specified memory address
3. Execution continues from that new address

10. Stop the Program

Actor – Program

Goal – The execution loop has exited cleanly and the program has terminated.

Steps:

1. The CPU reads opcode 43
2. The execution loop flag is set to False
3. The loop exits
4. The program terminates

11. Change Theme

Actor – User

Goal – Allow the user to change color theme from defaults to selected primary and secondary colors.

Steps:

1. User selects 'Change Theme' Button
2. System then prompts the user to select a primary color and press okay
3. System then prompts the user to select a secondary color and press okay
4. System then will refresh all widgets to be displayed in the color scheme

12. Truncation Overflow

Actor – System

Goal – Ensures that values/data remain within the 6-digit word size. Will cut off excess digits that are to the left of 6 digits for both positive and negative.

Steps:

1. System performs and completes a math operation
2. Truncation function is called on the accumulator
3. The accumulator is then truncated and returns the truncated value back to the accumulator system

13. Edit Table

Actor – User

Goal – Allow the user to change files in the GUI with COPY, PASTE, CUT, and ADD Options so files that may be missing a digit can be updated before running

Steps:

1. User selects a line in a loaded file
2. User selects if they want to COPY,CUT, or ADD
3. User then selects a different loaded line
4. User can then PASTE or ADD information

14. Save File

Actor – User

Goal – Allow the user to save loaded files in gui to anywhere in users cd, with its own specified name.

Steps:

1. User with already loaded file, clicks 'Save'
2. User is then prompted and asked where to save file and what name to give the file
3. When completed, the system will take items in the data frame for the gui and then create a txt file with each line individually added to the file

15. Reset

Actor – User

Goal – Allow the user to reset the gui and clear the output.

Steps:

1. User clicks 'Reset'
2. System then clears loaded files

3. System then clears the output
4. System prints message to output that the system was reset.